Race Condition

A race condition occurs when multiple processes or threads access shared resources concurrently, and the final outcome depends on the unpredictable timing or sequence of their execution. It's called a "race" because the threads are racing against each other to access/modify the shared data.

Key Characteristics

- Concurrent access to shared resources
- Unpredictable outcome based on execution order
- Non-deterministic behavior that may not always occur
- Difficult to debug due to timing-dependent nature

Example : E-commerce Inventory

python

```
inventory = {
   "laptop": 1  # Only 1 laptop in stock
}


# Customer A tries to buy laptop
def customer_a():
   if inventory["laptop"] > 0:
      process_payment("A")
      inventory["laptop"] -= 1


# Customer B also tries to buy laptop
def customer_b():
   if inventory["laptop"] > 0:
      process_payment("B")
      inventory["laptop"] -= 1
```

Race condition scenario:

- Both customers check inventory simultaneously
- Both see 1 laptop available
- Both process payments
- Both get confirmation emails
- **But only one laptop exists!**

Types of Race Conditions

1. Read-Modify-Write: Reading data, modifying it, and writing back (counter example)
2. Check-Then-Act: Checking a condition, then acting on it (banking example)
3. TOCTOU (Time of Check to Time of Use): Checking permissions, then using resource

Solution for Race Condition.

Synchronization

What is Synchronization?

Synchronization is the coordination of multiple entities (processes, threads, people, systems) to ensure they work together in an orderly manner, avoiding conflicts and maintaining data consistency.

Real-Life Examples

1. ATM Machine

- **Scenario**: One ATM, multiple customers
- **Synchronization**: People form a queue; only one person accesses the machine at a time
- **Key concept**: Resource locking and queue management

2. Airline Ticket Booking

- **Scenario**: Two agents trying to book the last seat on a flight
- **Synchronization**: First agent who confirms booking gets the seat; second gets "sold out"
- **Key concept**: Race condition prevention and data consistency

3. Dining at a Restaurant

- **Scenario**: Multiple waiters serving the same table
- **Synchronization**: Coordination to ensure orders aren't duplicated and food arrives correctly
- **Key concept**: Inter-process communication and coordination

OS-Based Examples

1. Printer Spooler

c

// Multiple processes trying to print simultaneously

Process A: "Print document1.pdf"

Process B: "Print document2.pdf"

Process C: "Print document3.pdf"

// Without synchronization: Mixed output

Output: doc1 doc2 doc3 (pages interleaved)

// With synchronization: Queue mechanism

Output: doc1 (all pages), then doc2, then doc3

Solution: Print queue with mutex locks

2. Bank Account Transaction

c

// Shared resource: Account balance = $1000

Thread 1: Withdraw $800

Thread 2: Withdraw $700

// Without synchronization:

Thread 1: Read balance (1000)

Thread 2: Read balance (1000)

Thread 1: Withdraw 800 ⟶ New balance: 200

Thread 2: Withdraw 700 ⟶ New balance: 300 ❌ (Incorrect)

// With synchronization (using mutex):

Thread 1: Lock account ⟶ Withdraw ⟶ Unlock

Thread 2: Wait for lock ⟶ Withdraw from updated balance ⟶ Correct result: -500

3. Producer-Consumer Problem

// Bounded buffer (queue) of size 5

Producer: Adds items to buffer

Consumer: Removes items from buffer

// Without synchronization:

Producer adds 6th item ⟶ Buffer overflow

Consumer removes from empty buffer ⟶ Underflow

// With synchronization (using semaphores):

Semaphore empty = 5 *// Empty slots*

Semaphore full = 0 *// Filled slots*

Producer:

wait(empty) *// Decrease empty count*

add_to_buffer()

signal(full) *// Increase full count*

Consumer:

wait(full) *// Wait if buffer empty*

remove_from_buffer()

signal(empty) *// Increase empty count*

4. Readers-Writers Problem

// Database with multiple readers and writers

// Without synchronization:

Writer 1 updates record A

Reader 1 reads record A (partial update)

Writer 2 updates record A → Data inconsistency

// With synchronization:

Readers can read simultaneously

Writers need exclusive access

When writer is writing, no readers can read

5. Dining Philosophers Problem

// 5 philosophers sitting around a table

// Each needs 2 forks to eat (shared with neighbors)

// Deadlock scenario:

All philosophers pick up left fork simultaneously

All wait for right fork indefinitely → System deadlock

// Solution:

// - Allow max 4 philosophers to sit simultaneously

// - Pick up both forks or none

// - Asymmetric solution (odd/even pickup order)

Common Synchronization Mechanisms in OS

1. **Mutex Locks** - Binary semaphore for mutual exclusion
2. **Semaphores** - Counting semaphores for resource management
3. **Monitors** - High-level synchronization construct
4. **Condition Variables** - For thread signaling
5. **Spinlocks** - Busy-waiting for short critical sections

Problems Solved by Synchronization

Problem	**Real-Life Example**	**OS Example**
Race Condition	Two people booking same seat	Two threads updating same variable
Deadlock	Cars blocking intersection	Processes holding resources circularly
Starvation	Always last in queue	Low-priority process never runs
Inconsistency	Bank balance errors	File corruption in concurrent writes

Critical Section Problem

The **Critical Section Problem** refers to the challenge of managing access to shared resources by multiple concurrent processes/threads, ensuring that only one process executes in its critical section at a time.

Definition

A **critical section** is a segment of code that accesses shared resources (variables, files, databases) that must not be executed by more than one process simultaneously.

The Four Requirements for a Solution

1. **Mutual Exclusion**: Only one process can execute in the critical section at a time

2. **Progress**: If no process is in the critical section, only processes waiting to enter can decide who enters next
3. **Bounded Waiting**: There's a limit on how long a process can wait to enter
4. **No Assumptions**: Solution should work regardless of CPU speed or number of processes

Real-World Analogy

Imagine a **public restroom with one toilet**:

- Multiple people need to use it (multiple processes)
- Only one person can use it at a time (mutual exclusion)
- Others must wait in line (waiting queue)
- When it's empty, the next person can enter (progress)
- No one should wait indefinitely (bounded waiting)

Code Example

The Problem (Without Protection)

```
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>

#define NUM_THREADS 5
#define NUM_ITERATIONS 100000

int shared_counter = 0;

void* increment_without_protection(void* arg) {
   int thread_id = *(int*)arg;
   for(int i = 0; i < NUM_ITERATIONS; i++) {
      // CRITICAL SECTION - UNSAFE
      int temp = shared_counter;  // Read
      temp++;                     // Modify
      shared_counter = temp;      // Write
```

```
        // REMAINDER SECTION
        usleep(1);  // Small delay to increase race probability
    }
    printf("Thread %d finished\n", thread_id);
    return NULL;
}

int main() {
    pthread_t threads[NUM_THREADS];
    int thread_ids[NUM_THREADS];

    // Create threads
    for(int i = 0; i < NUM_THREADS; i++) {
        thread_ids[i] = i;
        pthread_create(&threads[i], NULL, increment_without_protection, &thread_ids[i]);
    }
    // Wait for all threads
    for(int i = 0; i < NUM_THREADS; i++) {
        pthread_join(threads[i], NULL);
    }
    printf("Expected counter: %d\n", NUM_THREADS * NUM_ITERATIONS);
    printf("Actual counter: %d\n", shared_counter);
    // Will almost always be less than expected due to race conditions

    return 0;
}
```

Solution Using Mutex Locks

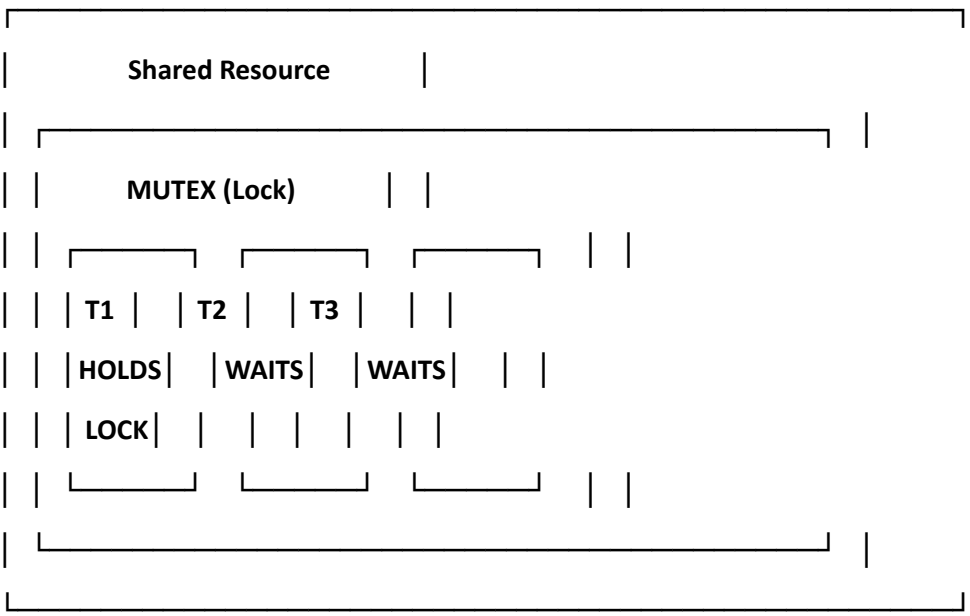
Mutex (Mutual Exclusion)

A mutex is a synchronization primitive used to protect shared resources from concurrent access. The name comes from mutual exclusion.

Basic Concept

A mutex acts like a lock that only one thread/process can hold at a time:

text



Key Operations

Operation	Purpose	Behavior
lock() / acquire()	Get exclusive access	Blocks if mutex is already held
unlock() / release()	Release exclusive access	Wakes up waiting threads
trylock()	Attempt to acquire	Returns immediately (non-blocking)

Examples

```
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>

#define NUM_THREADS 5
#define NUM_ITERATIONS 100000

int shared_counter = 0;
pthread_mutex_t mutex = PTHREAD_MUTEX_INITIALIZER;  // Mutex declaration

void* increment_with_mutex(void* arg) {
   int thread_id = *(int*)arg;

   for(int i = 0; i < NUM_ITERATIONS; i++) {
      // ENTRY SECTION - Acquire lock
      pthread_mutex_lock(&mutex);

      // CRITICAL SECTION - Protected access
      int temp = shared_counter;
      temp++;
      shared_counter = temp;

      // EXIT SECTION - Release lock
      pthread_mutex_unlock(&mutex);
      // REMAINDER SECTION
      usleep(1);
   }
   printf("Thread %d finished\n", thread_id);
   return NULL;
}
```

```
int main() {
    pthread_t threads[NUM_THREADS];
    int thread_ids[NUM_THREADS];

    // Initialize mutex
    pthread_mutex_init(&mutex, NULL);

    // Create threads
    for(int i = 0; i < NUM_THREADS; i++) {
        thread_ids[i] = i;
        pthread_create(&threads[i], NULL, increment_with_mutex, &thread_ids[i]);
    }

    // Wait for all threads
    for(int i = 0; i < NUM_THREADS; i++) {
        pthread_join(threads[i], NULL);
    }
    // Destroy mutex
    pthread_mutex_destroy(&mutex);

    printf("Expected counter: %d\n", NUM_THREADS * NUM_ITERATIONS);
    printf("Actual counter with mutex: %d\n", shared_counter);
    // Will always match expected value
    return 0;
}
```

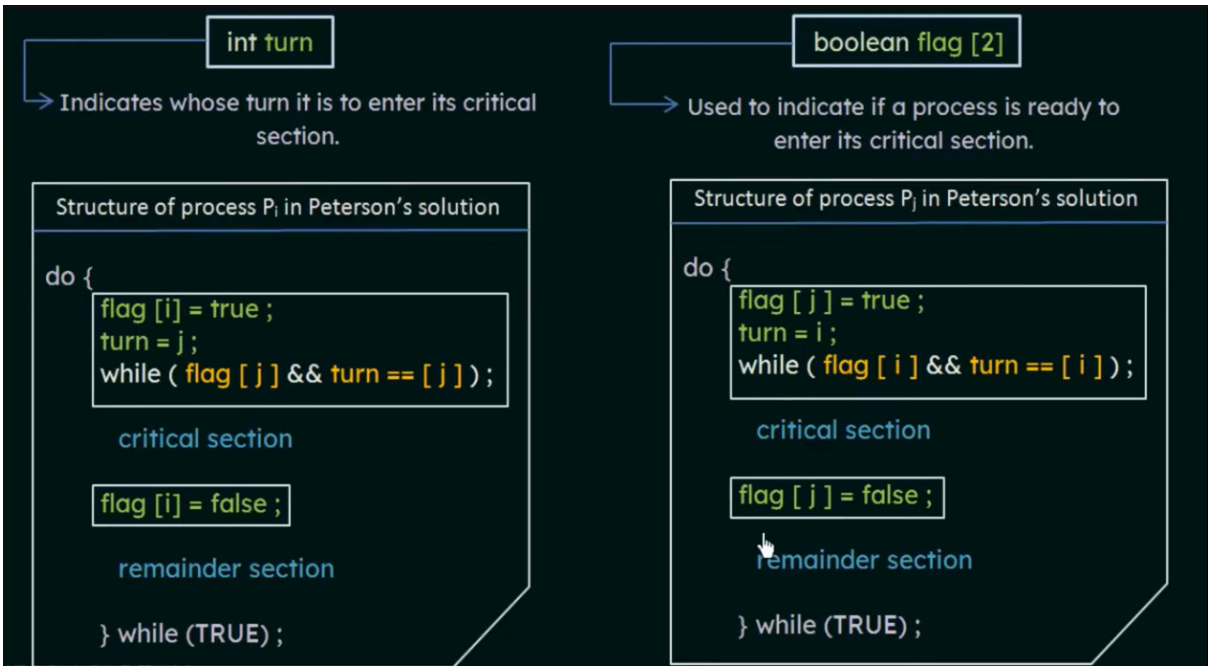
Mutex vs Peterson's Solution

Aspect	Peterson's Solution	Mutex
Type	Software-only	Hardware/OS-supported
Processes	Only 2	Unlimited
Waiting	Busy-waiting (wastes CPU)	Blocking (efficient)
Portability	Fails on modern CPUs	Works everywhere
Implementation	Educational	Production-ready

Peterson's Solution

Peterson's solution is a classic **software-based critical section solution** for process/thread synchronization. It works for **two processes** and guarantees:

- **Mutual Exclusion**
- **Progress**
- **Bounded Waiting**



How It Works

1. **flag[i] = TRUE** ⟶ Process i declares interest
2. **turn = j** ⟶ Process i gives priority to the other process

3. **while (flag[j] && turn == j)** → Wait if the other process wants to enter AND it's their turn
4. This ensures only one process enters the critical section

Correctness of Peterson's Solution

Peterson's solution is **correct** because it satisfies three critical properties for critical section problems:

---

1. Mutual Exclusion

Claim: Only one process can be in the critical section at a time.

Proof by Contradiction:

Assume both processes P0 and P1 are in the critical section simultaneously.

- For P0 to enter: flag[1] == FALSE **OR** turn == 0
- For P1 to enter: flag[0] == FALSE **OR** turn == 1

Since both are in CS, both flag[0] and flag[1] are **TRUE** (they set flags before entering).

Therefore:

- For P0: Since flag[1] is TRUE, we must have turn == 0
- For P1: Since flag[0] is TRUE, we must have turn == 1

But turn cannot be both 0 and 1 simultaneously → **Contradiction**

---

2. Progress

Claim: If no process is in CS and some processes want to enter, the selection cannot be postponed indefinitely.

Reasoning:

- If only one process wants to enter, it will set its flag and turn to the other, then check the while condition
- Since the other process doesn't want to enter (flag[j] == FALSE), it enters immediately
- If both want to enter, turn is set by whichever executed last
- The process with turn != its own index will be blocked, the other enters

---

3. Bounded Waiting

Claim: There is a limit on how many times other processes can enter before a waiting process gets its turn.

Proof:

- Suppose P1 is blocked in the while loop: while (flag[0] && turn == 0)
- P1 will wait only while **both** conditions are true
- P0 can enter CS at most **once** before releasing
- After P0 exits, it sets flag[0] = FALSE
- This breaks the while condition, allowing P1 to enter

Peterson's Solution in Modern Architecture

The Problem: Memory Model and Reordering

Peterson's solution **fails on modern architectures** due to:

Issue	**Explanation**
Compiler Reordering	Compiler may reorder instructions for optimization
CPU Out-of-Order Execution	Modern CPUs execute instructions speculatively
Memory Visibility	Writes may not be immediately visible to other cores
Weak Memory Models	x86 (TSO), ARM, PowerPC have different consistency guarantees

Example Failure Scenario

c

```
// Process 0
flag[0] = TRUE;
turn = 1;
while (flag[1] && turn == 1); // May be reordered!

// Process 1
flag[1] = TRUE;
turn = 0;
while (flag[0] && turn == 0); // May be reordered!
```

Due to reordering, both processes might simultaneously believe it's their turn, breaking mutual exclusion.

Modern Solutions

Approach	Description
Memory Barriers	smp_mb(), mfence, lwsync to enforce ordering
Atomic Operations	test_and_set(), compare_and_swap() (hardware-supported)
Mutex/Semaphores	OS-provided synchronization primitives
C11 Atomics	atomic_store(), atomic_load() with memory ordering

Corrected Modern Implementation (C11 Atomics)

c

```
#include <stdatomic.h>

atomic_bool flag[2];
atomic_int turn;

void peterson_modern(int i) {
   int j = 1 - i;

   atomic_store(&flag[i], true);
   atomic_store(&turn, j);
   atomic_thread_fence(memory_order_seq_cst);

   while (atomic_load(&flag[j]) && atomic_load(&turn) == j) {
      // Busy wait
   }

   // CRITICAL SECTION

   atomic_store(&flag[i], false);
}
```

The Problem: Compiler/CPU Reordering

Modern compilers and CPUs **reorder instructions** for optimization, breaking Peterson's delicate synchronization.

Original Peterson's Solution (P0)

c

```
// Process 0
flag[0] = 1;       // Statement A
turn = 1;          // Statement B

while (flag[1] == 1 && turn == 1);  // Statement C

// CRITICAL SECTION

flag[0] = 0;       // Statement D
```

Expected execution order: A → B → C

After Reordering (Compiler or CPU)

Case 1: Compiler Reordering

The compiler may swap **A** and **B** for optimization:

c

```
// Process 0 (after compiler optimization)
turn = 1;          // Statement B (moved up)
flag[0] = 1;       // Statement A (moved down)

while (flag[1] == 1 && turn == 1);  // Statement C

// CRITICAL SECTION

flag[0] = 0;
```

Case 2: CPU Out-of-Order Execution

Even if compiler preserves order, CPU may execute **B before A** speculatively.

Failure Scenario

Initial State: flag[0] = 0, flag[1] = 0, turn = 0

Step	P0 Execution	P1 Execution	State After
1	turn = 1 (B)	-	turn = 1
2	-	turn = 0 (B)	turn = 0
3	flag[0] = 1 (A)	-	flag[0] = 1
4	-	flag[1] = 1 (A)	flag[1] = 1
5	while (flag[1] && turn == 1)	-	Checks: flag[1]=1, turn=0 → **FALSE, enters CS**
6	-	while (flag[0] && turn == 0)	Checks: flag[0]=1, turn=0 → **TRUE, waits**
7	**IN CRITICAL SECTION**	-	P0 in CS
8	-	**IN CRITICAL SECTION**	P1 also enters!

Result: ❌ **Both processes in critical section simultaneously** → Mutual exclusion violated

---

Visual Comparison

Before Reordering (Correct)

text

P0: flag[0]=1 → turn=1 → [Check]

P1: flag[1]=1 → turn=0 → [Check]

Result: Only one enters ✓

After Reordering (Broken)

text

P0: turn=1 → flag[0]=1 → [Check]

P1: turn=0 → flag[1]=1 → [Check]

Result: Both can enter ✗

Why Reordering Happens

Level	Reason
Compiler	Optimizes for speed, assumes single-threaded semantics unless told otherwise
CPU	Executes independent instructions in parallel or out-of-order for efficiency
Memory	Writes may be buffered and not immediately visible to other cores

---

Fix: Memory Barriers

c

```
// Corrected with barriers
flag[0] = 1;
__sync_synchronize();  // Memory barrier (GCC)
turn = 1;
__sync_synchronize();

while (flag[1] == 1 && turn == 1);

// CRITICAL SECTION

flag[0] = 0;
```

Barrier ensures: All previous writes are visible before proceeding.

6.4.1 Memory Barriers (Fences)

The Problem: Modern CPUs and compilers optimize performance by reordering instructions. While a programmer might write flag = 1; after data = 100;, the CPU might execute the flag store first (because the cache line for flag is ready) and the data store second. On a multi-core system, another thread might see flag == 1 but still read stale (old) data.

What it is:
A memory barrier (or fence) is a low-level instruction that tells the CPU and compiler to enforce ordering constraints. It ensures that all load and store operations before the barrier are completed *before* any load or store operations after the barrier are executed.

Analogy:
Imagine a chef giving instructions to two assistants. Without a barrier, the assistants might execute instructions in whatever order is most convenient for them (e.g., "Pour the sauce" before "Plating the steak"). A memory barrier is like saying, "Do not do *any* of the instructions on this list until *all* the instructions on the previous list are 100% finished."

Types:

- Load Barrier: Ensures reads before the barrier finish before reads after the barrier.
- Store Barrier: Ensures writes before the barrier are visible before writes after the barrier.
- Full Barrier: Enforces both.

Why it matters: Atomic variables often implicitly include memory barriers to ensure that if one thread sets a flag atomically, all other data written before that flag is visible to the thread that sees the flag.

6.4.2 Hardware Instructions

The Problem: Memory barriers solve the *ordering* problem, but they do not solve the *race condition* problem (two threads modifying the same variable simultaneously). To solve that, we need instructions that can read, modify, and write a memory location without interruption.

What they are:
These are special CPU instructions that perform an operation atomically (indivisibly). The CPU guarantees that no other core or process can access that memory location while the instruction is executing.

Key Hardware Instructions:

Instruction	**Behavior**
Test-and-Set (TAS)	**Reads a value, sets it to true (1), and returns the original value. All in one step.**
Compare-and-Swap (CAS)	**Compares a variable to an expected value; if equal, replaces it with a new value; returns the original value. (The most versatile primitive.)**
Fetch-and-Add (FAA)	**Atomically increments a variable and returns the old value.**

How they work:
On x86 architectures, these are typically implemented using the LOCK prefix (e.g., LOCK XCHG, LOCK CMPXCHG). This prefix tells the processor to lock the memory bus or cache line to prevent other cores from accessing it until the operation completes.

Why they matter: These instructions are the actual "teeth" of synchronization. Every mutex, semaphore, and atomic variable ultimately relies on one of these hardware instructions at the very bottom of the stack.

6.4.3 Atomic Variables

The Problem: While hardware instructions (like CAS) are powerful, they are difficult for programmers to use directly. They require assembly language or compiler intrinsics and are architecture-specific (x86, ARM, and RISC-V have different implementations). Programmers need a safe, portable abstraction.

What they are:
Atomic variables are a programming language or OS-level abstraction that wraps the hardware instructions (6.4.2) and memory barriers (6.4.1) into a simple API.

Instead of writing assembly to perform a LOCK CMPXCHG, a programmer can write:

c

// C11 Standard

atomic_int counter;

atomic_fetch_add(&counter, 1);

What they provide:

1. Abstraction: The same code works on x86, ARM, and RISC-V. The compiler inserts the appropriate hardware instructions and barriers.
2. Lock-Free Operations: They allow updates to shared variables without using heavy OS mutexes (which involve context switching).
3. Implicit Barriers: Depending on the "memory order" specified (e.g., memory_order_seq_cst vs. memory_order_relaxed), the compiler automatically inserts the necessary memory barriers (from 6.4.1) to ensure visibility across threads.

Semaphores

A semaphore is a synchronization primitive that uses a counter to control access to shared resources. Unlike mutexes, semaphores can allow multiple threads to access a resource concurrently.

Basic Concept

A semaphore maintains a counter and supports two atomic operations:

Operation	**Also Called**	**Behavior**
wait()	P(), down(), acquire()	Decrements counter; blocks if counter becomes negative

signal()	V(), up(), release()	Increments counter; wakes up waiting thread

---

Types of Semaphores

Type	Counter Range	Purpose
Binary Semaphore	0 or 1	Acts like a mutex (mutual exclusion)
Counting Semaphore	0 to N	Controls access to multiple identical resources

---

1. Binary Semaphore (Mutual Exclusion)

c

```
#include <stdio.h>
#include <pthread.h>
#include <semaphore.h>

sem_t mutex;  // Binary semaphore (acts like mutex)
int shared_counter = 0;

void* increment(void* arg) {
    for (int i = 0; i < 1000000; i++) {
        sem_wait(&mutex);       // P() operation
        shared_counter++;       // Critical section
        sem_post(&mutex);       // V() operation
    }
    return NULL;
}

int main() {
    pthread_t t1, t2;

    // Initialize binary semaphore with value 1
    sem_init(&mutex, 0, 1);  // 0 = thread-shared, 1 = initial value
```

```
    pthread_create(&t1, NULL, increment, NULL);
    pthread_create(&t2, NULL, increment, NULL);
    pthread_join(t1, NULL);
    pthread_join(t2, NULL);

    printf("Counter: %d\n", shared_counter);  // Always 2000000

    sem_destroy(&mutex);
    return 0;
}
```

---

2. Counting Semaphore (Resource Pool)

Scenario: 3 printers available, 10 processes need to print

```
#include <stdio.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>

sem_t printers;  // Counting semaphore
int available_printers = 3;

void* use_printer(void* arg) {
    int id = *(int*)arg;

    printf("Process %d: Waiting for printer...\n", id);
    sem_wait(&printers);  // Decrease available count

    // Critical section - using printer
    printf("Process %d: Acquired printer. Printing...\n", id);
    sleep(2);  // Simulate printing time
    printf("Process %d: Done printing.\n", id);
```

```
    sem_post(&printers);  // Increase available count
    return NULL;
}

int main() {
    pthread_t processes[10];
    int ids[10];

    // Initialize semaphore with 3 available resources
    sem_init(&printers, 0, 3);

    // Create 10 processes
    for (int i = 0; i < 10; i++) {
        ids[i] = i;
        pthread_create(&processes[i], NULL, use_printer, &ids[i]);
    }

    // Wait for all to complete
    for (int i = 0; i < 10; i++) {
        pthread_join(processes[i], NULL);
    }

    sem_destroy(&printers);
    return 0;
}
```

These operations must not be interrupted mid-way.

✅ **b) Sequential Memory Consistency**

- The system must follow **sequential consistency**:

All operations appear in the order written in the program.

- Without this, **instruction reordering** (by CPU or compiler) can break Peterson's logic.

✅ **c) Visibility of Shared Variables**

- Changes made by one process must be **immediately visible** to the other process.
- This requires:
  - Cache coherence
  - Proper memory synchronization

⚠️ **Problem with Modern Hardware**

Modern CPUs use:

- **Out-of-order execution**
- **Cache optimization**
- **Instruction reordering**

👉 This can break Peterson's algorithm unless additional hardware instructions are used.

🔧 **2. Hardware-Level Fix (Memory Barriers)**

To make Peterson's solution work on real hardware, we use:

🔹 **Memory Fence / Barrier Instructions**

- Prevent reordering of instructions.
- Ensure correct visibility between processes.

Example (conceptual):

```
flag[i] = true;
```

FROM CHATGPT Solution

🔧 2. Hardware-Level Fix (Memory Barriers)

To make Peterson's solution work on real hardware, we use:

- **Memory Fence / Barrier Instructions**
  - Prevent reordering of instructions.
  - Ensure correct visibility between processes.

Example (conceptual):

```
flag[i] = true;
memory_fence();   // hardware instruction
turn = j;
```

⚙️ 3. Alternative: Pure Hardware Solutions

Instead of Peterson's solution, hardware provides **direct mutual exclusion primitives**:

- **Test-and-Set Instruction**

```
while (TestAndSet(lock)) ;
```

- **Compare-and-Swap (CAS)**

```
CAS(lock, 0, 1);
```

- **Swap Instruction**
  - Atomically swaps values of variables

👉 These are **true hardware solutions**, unlike Peterson's algorithm.

Producer-Consumer Problem

Definition

The **Producer-Consumer Problem** (also called Bounded-Buffer Problem) involves two types of processes sharing a fixed-size buffer. Producers generate data and put it into the buffer, while consumers take data from the buffer.

Key Challenges

- Producer must not add data to a **full buffer**
- Consumer must not remove data from an **empty buffer**
- Access to buffer must be **mutually exclusive**

Visual Representation

text

Producer(s) → [BUFFER] → Consumer(s)

Fixed Size

Put data Remove data

Simple C Code Example

c

```
#include <stdio.h>
#include <pthread.h>
#include <semaphore.h>

#define BUFFER_SIZE 5
#define NUM_ITEMS 10

int buffer[BUFFER_SIZE];
int count = 0;  // Items in buffer
int in = 0;     // Next free position
int out = 0;    // Next full position

sem_t empty;    // Counts empty slots
sem_t full;     // Counts full slots
pthread_mutex_t mutex;  // Protects buffer access
```

```
void* producer(void* arg) {
    for(int i = 0; i < NUM_ITEMS; i++) {
        // Wait for empty slot
        sem_wait(&empty);

        // Lock buffer
        pthread_mutex_lock(&mutex);

        // CRITICAL SECTION: Add item
        buffer[in] = i;
        printf("Produced: %d at position %d\n", i, in);
        in = (in + 1) % BUFFER_SIZE;
        count++;

        // Unlock buffer
        pthread_mutex_unlock(&mutex);

        // Signal that buffer has data
        sem_post(&full);
    }
    return NULL;
}

void* consumer(void* arg) {
    for(int i = 0; i < NUM_ITEMS; i++) {
        // Wait for full slot
        sem_wait(&full);

        // Lock buffer
        pthread_mutex_lock(&mutex);
```

```
        // CRITICAL SECTION: Remove item
        int item = buffer[out];
        printf("Consumed: %d from position %d\n", item, out);
        out = (out + 1) % BUFFER_SIZE;
        count--;

        // Unlock buffer
        pthread_mutex_unlock(&mutex);

        // Signal that buffer has empty slot
        sem_post(&empty);
    }
    return NULL;
}

int main() {
    pthread_t prod, cons;

    sem_init(&empty, 0, BUFFER_SIZE);  // Initially all slots empty
    sem_init(&full, 0, 0);             // Initially no slots full
    pthread_mutex_init(&mutex, NULL);

    pthread_create(&prod, NULL, producer, NULL);
    pthread_create(&cons, NULL, consumer, NULL);

    pthread_join(prod, NULL);
    pthread_join(cons, NULL);

    return 0;
}
```

Real-World Examples

- **Print spooler**: Multiple apps produce print jobs, printer consumes them
- **Web servers**: Request queue between web servers and application servers
- **Streaming**: Video buffer between producer (streaming server) and consumer (viewer)

---

2. Readers-Writers Problem

Definition

The **Readers-Writers Problem** involves a shared data resource where:

- **Readers** only read the data (don't modify)
- **Writers** modify the data
- Multiple readers can access simultaneously
- Only one writer can access at a time
- No reader should access while a writer is writing

Key Challenges

- **Reader priority**: If readers keep coming, writers may starve
- **Writer priority**: Once a writer is ready, no new readers should start
- **Fairness**: Balance between readers and writers

Visual Representation

text

Readers (R1, R2, R3) → Can read simultaneously

↓

[SHARED DATA]

↓

Writers (W1, W2) → Only one can write at a time

Simple C Code Example (Reader Priority)

c

```
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>

pthread_mutex_t mutex = PTHREAD_MUTEX_INITIALIZER;
pthread_mutex_t write_lock = PTHREAD_MUTEX_INITIALIZER;
```

```
int reader_count = 0;
int shared_data = 1000;  // Shared resource

void* reader(void* arg) {
   int id = *(int*)arg;

   for(int i = 0; i < 3; i++) {
      // ENTRY SECTION: Update reader count
      pthread_mutex_lock(&mutex);
      reader_count++;
      if(reader_count == 1) {
         pthread_mutex_lock(&write_lock);  // First reader locks writers out
      }
      pthread_mutex_unlock(&mutex);

      // CRITICAL SECTION: Reading
      printf("Reader %d reads: %d (active readers: %d)\n",
            id, shared_data, reader_count);
      sleep(1);  // Simulate reading time

      // EXIT SECTION: Decrease reader count
      pthread_mutex_lock(&mutex);
      reader_count--;
      if(reader_count == 0) {
         pthread_mutex_unlock(&write_lock);  // Last reader allows writers
      }
      pthread_mutex_unlock(&mutex);

      sleep(rand() % 2);
   }
   return NULL;
```

```
}

void* writer(void* arg) {
    int id = *(int*)arg;

    for(int i = 0; i < 2; i++) {
        // ENTRY SECTION: Wait for exclusive access
        pthread_mutex_lock(&write_lock);

        // CRITICAL SECTION: Writing
        shared_data += 100;
        printf("Writer %d writes: %d\n", id, shared_data);
        sleep(2);  // Simulate writing time

        // EXIT SECTION: Release access
        pthread_mutex_unlock(&write_lock);

        sleep(rand() % 3);
    }
    return NULL;
}

int main() {
    pthread_t readers[3], writers[2];
    int reader_ids[3] = {1, 2, 3};
    int writer_ids[2] = {1, 2};

    // Create threads
    for(int i = 0; i < 3; i++)
        pthread_create(&readers[i], NULL, reader, &reader_ids[i]);
    for(int i = 0; i < 2; i++)
```

```
        pthread_create(&writers[i], NULL, writer, &writer_ids[i]);


    // Wait for completion
    for(int i = 0; i < 3; i++)
        pthread_join(readers[i], NULL);
    for(int i = 0; i < 2; i++)
        pthread_join(writers[i], NULL);


    return 0;
}
```

Variations

1. First Readers-Writers Problem (Reader Priority)

- Readers don't wait unless a writer has already obtained write lock
- Can cause writer starvation

2. Second Readers-Writers Problem (Writer Priority)

- Once a writer is ready, no new readers can start
- Prevents writer starvation but can cause reader starvation

3. Third Readers-Writers Problem (Fairness)

- Uses a queue to ensure alternating access
- No starvation for either readers or writers

Real-World Examples

- **Database systems**: Multiple users reading, few updating
- **File systems**: Multiple processes reading, few writing
- **Configuration management**: Many readers, occasional updates
- **Cache systems**: Multiple reads, periodic cache updates

Liveness in Threading

Liveness refers to the ability of a concurrent program to **make progress** and eventually complete execution. When liveness fails, threads become stuck and cannot proceed.

Types of Liveness Failures

Failure	Description	Severity
Deadlock	Threads block forever waiting for each other	Critical
Starvation	Thread never gets necessary resources	High
Livelock	Threads keep changing state but make no progress	Moderate
Priority Inversion	Low-priority thread blocks high-priority thread	Moderate

---

1. Deadlock

Definition

Two or more threads are blocked forever, each waiting for a resource held by another.

Necessary Conditions (Coffman Conditions)

1. **Mutual Exclusion**: Resources cannot be shared
2. **Hold and Wait**: Thread holds resources while waiting for others
3. **No Preemption**: Resources cannot be forcibly taken
4. **Circular Wait**: Circular chain of threads waiting for each other

Deadlock Example

c

```
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>

pthread_mutex_t lock1 = PTHREAD_MUTEX_INITIALIZER;
pthread_mutex_t lock2 = PTHREAD_MUTEX_INITIALIZER;

void* thread1(void* arg) {
    printf("Thread 1: Trying to acquire lock1\n");
```

```
    pthread_mutex_lock(&lock1);
    printf("Thread 1: Acquired lock1\n");
    sleep(1);  // Give thread2 time to acquire lock2

    printf("Thread 1: Trying to acquire lock2\n");
    pthread_mutex_lock(&lock2);  // Deadlock! Waiting for lock2
    printf("Thread 1: Acquired lock2\n");

    pthread_mutex_unlock(&lock2);
    pthread_mutex_unlock(&lock1);
    return NULL;
}

void* thread2(void* arg) {
    printf("Thread 2: Trying to acquire lock2\n");
    pthread_mutex_lock(&lock2);
    printf("Thread 2: Acquired lock2\n");
    sleep(1);  // Give thread1 time to acquire lock1

    printf("Thread 2: Trying to acquire lock1\n");
    pthread_mutex_lock(&lock1);  // Deadlock! Waiting for lock1
    printf("Thread 2: Acquired lock1\n");

    pthread_mutex_unlock(&lock1);
    pthread_mutex_unlock(&lock2);
    return NULL;
}

int main() {
    pthread_t t1, t2;
```

```
    pthread_create(&t1, NULL, thread1, NULL);
    pthread_create(&t2, NULL, thread2, NULL);

    pthread_join(t1, NULL);  // Never returns - deadlock
    pthread_join(t2, NULL);

    return 0;
}
```

Visual Representation:

text

Thread 1: Holds lock1 → Wants lock2 → ⏸️ Blocked

Thread 2: Holds lock2 → Wants lock1 → ⏸️ Blocked

Result: Both threads stuck forever ❌

Deadlock Prevention

Strategy	Implementation
Lock Ordering	Always acquire locks in same order
Timeout	Use pthread_mutex_timedlock()
Try-Lock	Use pthread_mutex_trylock() and backoff
Avoid Hold & Wait	Acquire all locks at once

c

```
// Solution: Fixed lock ordering
void* thread1_fixed(void* arg) {
    pthread_mutex_lock(&lock1);  // Always lock1 first
    pthread_mutex_lock(&lock2);  // Then lock2
    // Critical section
    pthread_mutex_unlock(&lock2);
    pthread_mutex_unlock(&lock1);
    return NULL;
```

```
}

void* thread2_fixed(void* arg) {
    pthread_mutex_lock(&lock1);  // Same order!
    pthread_mutex_lock(&lock2);
    // Critical section
    pthread_mutex_unlock(&lock2);
    pthread_mutex_unlock(&lock1);
    return NULL;
}
```

---

2. Starvation

Definition

A thread never gets access to required resources because other threads monopolize them.

Starvation Example

c

```
#include <stdio.h>
#include <pthread.h>
#include <semaphore.h>

sem_t resource;
int low_priority_count = 0;
int high_priority_count = 0;

void* high_priority_thread(void* arg) {
    int id = *(int*)arg;
    while (1) {
        sem_wait(&resource);
        high_priority_count++;
        printf("High priority %d: Running (High: %d, Low: %d)\n",
               id, high_priority_count, low_priority_count);
```

```
        sem_post(&resource);
        // Never yields - starves low priority threads
    }
    return NULL;
}

void* low_priority_thread(void* arg) {
    int id = *(int*)arg;
    while (1) {
        sem_wait(&resource);
        low_priority_count++;
        printf("Low priority %d: Running (High: %d, Low: %d)\n",
               id, high_priority_count, low_priority_count);
        sem_post(&resource);
        // Rarely gets to run
    }
    return NULL;
}

int main() {
    pthread_t high[3], low[2];
    int ids_high[3] = {1, 2, 3};
    int ids_low[2] = {1, 2};

    sem_init(&resource, 0, 1);

    // Create high priority threads
    for (int i = 0; i < 3; i++) {
        pthread_create(&high[i], NULL, high_priority_thread, &ids_high[i]);
    }
```

```
    // Create low priority threads
    for (int i = 0; i < 2; i++) {
        pthread_create(&low[i], NULL, low_priority_thread, &ids_low[i]);
    }

    pthread_join(high[0], NULL);  // Never returns
    return 0;
}
```

Starvation Prevention

Technique	Description
Fair Scheduling	Use fair locks (e.g., FIFO)
Priority Aging	Increase priority of waiting threads over time
Resource Limits	Limit how long a thread can hold resources
Yield	Threads voluntarily yield CPU

c

```
// Fair semaphore implementation
#include <stdatomic.h>
#include <unistd.h>

typedef struct {
    atomic_int counter;
    pthread_cond_t cond;
    pthread_mutex_t mutex;
} fair_semaphore;

void fair_wait(fair_semaphore* sem) {
    pthread_mutex_lock(&sem->mutex);
    while (atomic_load(&sem->counter) == 0) {
        pthread_cond_wait(&sem->cond, &sem->mutex);
    }
    atomic_fetch_sub(&sem->counter, 1);
```

```
    pthread_mutex_unlock(&sem->mutex);
}

void fair_signal(fair_semaphore* sem) {
    pthread_mutex_lock(&sem->mutex);
    atomic_fetch_add(&sem->counter, 1);
    pthread_cond_signal(&sem->cond);  // Wakes waiting threads in order
    pthread_mutex_unlock(&sem->mutex);
}
```

---

3. Livelock

Definition

Threads continuously change state in response to each other but make no actual progress.

Livelock Example

c

```
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>
#include <stdlib.h>

pthread_mutex_t lock1 = PTHREAD_MUTEX_INITIALIZER;
pthread_mutex_t lock2 = PTHREAD_MUTEX_INITIALIZER;

void* thread1_livelock(void* arg) {
    int retries = 0;
    while (retries < 10) {
        printf("Thread 1: Trying to acquire locks (attempt %d)\n", retries);

        pthread_mutex_lock(&lock1);
        usleep(100000);  // Simulate work
```

```
        if (pthread_mutex_trylock(&lock2) != 0) {
            printf("Thread 1: Can't get lock2, releasing lock1\n");
            pthread_mutex_unlock(&lock1);
            retries++;
            usleep(200000);  // Backoff and retry
            continue;
        }

        // Critical section
        printf("Thread 1: Acquired both locks!\n");
        pthread_mutex_unlock(&lock2);
        pthread_mutex_unlock(&lock1);
        break;
    }
    return NULL;
}

void* thread2_livelock(void* arg) {
    int retries = 0;
    while (retries < 10) {
        printf("Thread 2: Trying to acquire locks (attempt %d)\n", retries);

        pthread_mutex_lock(&lock2);
        usleep(100000);  // Simulate work

        if (pthread_mutex_trylock(&lock1) != 0) {
            printf("Thread 2: Can't get lock1, releasing lock2\n");
            pthread_mutex_unlock(&lock2);
            retries++;
            usleep(200000);  // Backoff and retry
            continue;
```

```
        }

        // Critical section
        printf("Thread 2: Acquired both locks!\n");
        pthread_mutex_unlock(&lock1);
        pthread_mutex_unlock(&lock2);
        break;
    }
    return NULL;
}

int main() {
    pthread_t t1, t2;

    pthread_create(&t1, NULL, thread1_livelock, NULL);
    pthread_create(&t2, NULL, thread2_livelock, NULL);

    pthread_join(t1, NULL);
    pthread_join(t2, NULL);

    printf("Both threads finished (may have livelocked)\n");
    return 0;
}
```

Visual Representation:

text

Thread 1: Acquire lock1 → Can't get lock2 → Release lock1 → Retry

Thread 2: Acquire lock2 → Can't get lock1 → Release lock2 → Retry

Result: Both keep retrying but never succeed ♻️

Livelock vs Deadlock

Aspect	**Deadlock**	**Livelock**

State	Threads blocked	Threads active
CPU Usage	0% (blocked)	100% (busy)
Progress	None	None (but trying)
Detection	Easier (timeout)	Harder (appears active)

Livelock Prevention

Strategy	**Implementation**
Exponential Backoff	Increase delay between retries
Randomized Delays	Use random sleep times
Deterministic Order	Use fixed lock ordering
Timeouts	Fail after maximum attempts

c

```
// Solution: Exponential backoff with random delay
void* thread_fixed(void* arg) {
    int retries = 0;
    int delay = 10000;  // Start with 10ms

    while (retries < 10) {
        pthread_mutex_lock(&lock1);

        if (pthread_mutex_trylock(&lock2) == 0) {
            // Success!
            printf("Acquired locks!\n");
            pthread_mutex_unlock(&lock2);
            pthread_mutex_unlock(&lock1);
            break;
        }
```

```
        pthread_mutex_unlock(&lock1);

        // Exponential backoff with randomness
        usleep(delay + rand() % delay);
        delay *= 2;  // Double delay each time
        retries++;
    }
    return NULL;
}
```